MANUAL TÉCNICO

Visualizador Excel Pro

Fusion Edition

Versión	1.0.0
Fecha de Emisión	Marzo 2026
Clasificación	Documentación Técnica Interna
Plataforma	Web – Navegador moderno (HTML5)
Idioma	Español

© 2026 Visualizador Pro | Analítica de Datos Simplificada

Tabla de Contenidos

Tabla de Contenidos 2
1. Introducción 4
1.1 Descripción General del Sistema 4
1.2 Propósito del Documento 4
1.3 Alcance 4
1.4 Convenciones del Documento 4
2. Arquitectura del Sistema 5
2.1 Tecnologías y Dependencias 5
2.2 Estructura del Archivo Fuente 5
2.3 Modelo de Ejecución 5
3. Componentes de la Interfaz de Usuario 7
3.1 Encabezado (Header) 7
3.2 Zona de Carga de Archivos (Drop Zone) 7
3.2.1 Drag & Drop 7
3.2.2 Selector de Archivos Tradicional 7
3.3 Panel de Información del Archivo (File Info) 7
3.4 Selector de Hoja (Sheet Selection) 7
3.5 Panel de Resultados (Charts Section) 7
3.6 Overlay de Carga 8
4. Lógica de Procesamiento de Datos 9
4.1 Función processFile(file) 9
4.2 Función renderCharts(sheetName) 9
4.3 Detección Automática de Columnas 9
4.4 Función aggregateData(data, catCol, numCol) 10
4.5 Función getTopN(agg, n) 10
4.6 Función createHistogram(data, col) 10
4.7 Función generateColors(n) 10
5. Motor de Visualización 11
5.1 Función createChart(id, type, labels, values, title, color, dlConfig, fill) 11
5.2 Catálogo de Gráficas 11
5.2.1 Gráfico de Barras (Bar Chart) 11
5.2.2 Gráfico de Pastel (Pie Chart) 11
5.2.3 Gráfico de Línea (Line Chart) 11
5.2.4 Histograma 11

5.2.5 Gráfico de Anillo (Doughnut Chart) 11
5.2.6 Gráfico de Área (Area Chart)....11
5.3 Configuración Compartida de Etiquetas (commonDatalabels)....12
6. Formatos de Archivo Soportados....13
6.1 Especificaciones de Compatibilidad....13
6.2 Limitaciones de Archivo....13
6.3 Procesamiento Interno por SheetJS....13
7. Rendimiento y Limitaciones Conocidas....14
7.1 Comportamiento con Grandes Volúmenes de Datos....14
7.2 Limitaciones Técnicas....14
7.3 Compatibilidad de Navegadores....14
8. Guía de Mantenimiento y Extensibilidad....15
8.1 Actualización de Dependencias....15
8.2 Agregar un Nuevo Tipo de Gráfico....15
8.3 Soporte Multi-Columna Numérica....15
8.4 Mejoras Recomendadas de Seguridad y UX....15
9. Glosario de Términos....16
10. Historial de Cambios....17

1. Introducción

1.1 Descripción General del Sistema

Visualizador Excel Pro (Fusion Edition) es una aplicación web de análisis y visualización de datos diseñada para transformar archivos tabulares — en formatos XLSX, XLS y CSV — en representaciones gráficas interactivas, sin necesidad de instalación ni configuración de servidor.

La herramienta opera completamente en el navegador del cliente mediante tecnologías HTML5, JavaScript ES6+ y Canvas API, lo que garantiza privacidad total de los datos del usuario, ya que ningún archivo es transmitido a servidores externos.

1.2 Propósito del Documento

Este manual técnico proporciona documentación exhaustiva sobre la arquitectura, funcionalidades, lógica interna y pautas de mantenimiento del software Visualizador Excel Pro. Está dirigido a:

- Desarrolladores responsables del mantenimiento y evolución del sistema
- Técnicos de soporte que deban diagnosticar comportamientos inesperados
- Auditores técnicos que evalúen la calidad y seguridad del código
- Usuarios avanzados con conocimiento de desarrollo web

1.3 Alcance

El presente documento cubre:

- Arquitectura tecnológica y dependencias del sistema
- Flujo de procesamiento de archivos
- Motor de detección automática de columnas
- Motor de generación de visualizaciones (6 tipos de gráfica)
- Estructura del DOM y comportamiento de la interfaz
- Limitaciones conocidas y recomendaciones de mejora

1.4 Convenciones del Documento

Convención	Significado
CÓDIGO	Fragmento de código fuente JavaScript o HTML
[Nota]	Aclaración relevante para el desarrollador
[Advertencia]	Comportamiento que puede causar errores si se modifica
→	Flujo de ejecución o resultado esperado

2. Arquitectura del Sistema

2.1 Tecnologías y Dependencias

El sistema está construido como una Single Page Application (SPA) basada íntegramente en HTML, CSS y JavaScript, sin frameworks de servidor. Las dependencias externas se cargan mediante CDN públicas:

Librería	Versión	Función
Tailwind CSS	CDN Latest	Sistema de utilidades CSS para el diseño UI
Font Awesome	6.5.1	Iconografía vectorial (SVG icon font)
SheetJS (xlsx)	0.18.5	Lectura y parseo de archivos .xlsx / .xls / .csv
Chart.js	4.4.1	Motor de renderizado de gráficas mediante Canvas API
chartjs-plugin-datalabels	2.2.0	Superposición de etiquetas de valor sobre gráficas
Google Fonts (Inter)	Variable	Tipografía principal de la interfaz

2.2 Estructura del Archivo Fuente

El sistema se distribuye como un único archivo index.html autocontenido, organizado en los siguientes bloques lógicos:

```
index.html
  ├── <head>
  │     ├── Meta tags y viewport
  │     ├── CDN: Tailwind CSS, Font Awesome
  │     ├── CDN: SheetJS, Chart.js, chartjs-plugin-datalabels
  │     └── <style id="app-style">  — estilos locales
  └── <body>
        ├── <header>  — Barra de título
        ├── <main>
        │     ├── #dropZone  — Zona de carga de archivos
        │     ├── #fileInfo  — Panel de metadatos del archivo
        │     ├── #sheetSelection  — Selector de hoja
        │     ├── #chartsSection   — Cuadrícula de gráficas
        │     └── #loadingOverlay  — Overlay de progreso
        ├── <footer>
        └── <script>  — Lógica de aplicación (JS)
```

2.3 Modelo de Ejecución

La aplicación sigue un modelo de ejecución enteramente client-side, sin estado persistente entre sesiones. La siguiente secuencia describe el ciclo de vida completo de una sesión de uso:

1. El usuario abre index.html en el navegador.
2. Se registra el plugin ChartDataLabels en Chart.js.

3. Se adjuntan los listeners de eventos al dropZone y fileInput.
4. El usuario sube un archivo → se dispara processFile().
5. SheetJS parsea el archivo binario a objeto workbook en memoria.
6. Se puebla el selector de hojas (sheetSelector) con los SheetNames.
7. Se invoca automáticamente renderCharts() con la primera hoja.
8. La función agrega, detecta columnas y crea los 6 objetos Chart.
9. El panel de resultados se hace visible.
10. El usuario puede cambiar de hoja o limpiar la sesión (location.reload()).

3. Componentes de la Interfaz de Usuario

3.1 Encabezado (Header)

El encabezado presenta el nombre del sistema sobre un fondo con degradado de azul profundo a índigo (from-blue-700 to-indigo-800). Contiene:

- Título principal: "Visualizador Excel Pro" (texto blanco, 4xl, extrabold)
- Subtítulo descriptivo: "Análisis inteligente de datos con visualización en tiempo real"

Este componente es puramente decorativo e informativo; no contiene lógica funcional.

3.2 Zona de Carga de Archivos (Drop Zone)

La zona de carga es el punto de entrada principal del sistema. Implementa dos mecanismos de selección de archivos:

3.2.1 Drag & Drop

Eventos gestionados directamente sobre el elemento #dropZone:

- ondragover: Previene el comportamiento por defecto del navegador y activa la clase CSS .active (realce visual azul).
- ondragleave: Desactiva la clase .active al salir del área.
- ondrop: Extrae el primer archivo de e.dataTransfer.files y llama processFile().

3.2.2 Selector de Archivos Tradicional

Un elemento <input type="file" id="fileInput"> oculto es activado mediante el evento onclick del dropZone. El filtro de tipo de archivo acepta exclusivamente .xlsx, .xls y .csv.

3.3 Panel de Información del Archivo (File Info)

Tras la carga exitosa, el panel #fileInfo se hace visible mostrando:

- Nombre del archivo (elemento #fileName)
- Tamaño en kilobytes, formateado con un decimal (elemento #fileSize)
- Botón de limpieza (#clearButton) que ejecuta location.reload() para reiniciar la sesión

3.4 Selector de Hoja (Sheet Selection)

El panel #sheetSelection contiene un elemento <select id="sheetSelector"> cuyos options son generados dinámicamente a partir del array workbook.SheetNames. El botón "Actualizar Visualización" invoca renderCharts() con el nombre de hoja actualmente seleccionado.

3.5 Panel de Resultados (Charts Section)

El panel #chartsSection contiene el grid de 6 tarjetas de gráfica. Cada tarjeta (.chart-card) incluye:

- Un título descriptivo fijo
- Un elemento <canvas> con ID único, referenciado por Chart.js

La disposición del grid es responsive:

- Pantallas < 768px: 1 columna
- Pantallas 768px–1279px: 2 columnas
- Pantallas >= 1280px: 3 columnas

3.6 Overlay de Carga

El elemento #loadingOverlay es un overlay de posición fija con fondo semitransparente y efecto backdrop-filter: blur(4px). Se activa mediante showLoading(msg) y se desactiva con hideLoading(). Muestra un ícono animado con rotación CSS (@keyframes spin) y el mensaje de estado actual.

4. Lógica de Procesamiento de Datos

4.1 Función processFile(file)

Esta función es el punto de entrada de la lógica de negocio. Recibe un objeto File del DOM y ejecuta:

11. Activa el overlay de carga con el mensaje "Analizando estructura del archivo..."
12. Crea un objeto FileReader y lee el archivo como ArrayBuffer.
13. En el callback onload, convierte el buffer a Uint8Array y lo pasa a XLSX.read().
14. Actualiza los elementos #fileName y #fileSize en el DOM.
15. Muestra los paneles #fileInfo y #sheetSelection.
16. Puebla el selector de hojas con los SheetNames del workbook.
17. Llama automáticamente a renderCharts() con la primera hoja.

[Advertencia] Si el archivo no es un workbook válido, XLSX.read() lanzará una excepción capturada por el bloque try/catch, que muestra un alert con el mensaje de error y llama hideLoading().

4.2 Función renderCharts(sheetName)

Esta función orquesta la generación completa de visualizaciones para una hoja dada:

18. Activa el overlay de carga.
19. Convierte la hoja a array de objetos JSON mediante XLSX.utils.sheet_to_json().
20. Si el array está vacío, emite un alert y termina.
21. Detecta automáticamente las columnas de trabajo (ver sección 4.3).
22. Destruye todos los objetos Chart previos en chartInstances para evitar memory leaks.
23. Hace visible el panel #chartsSection.
24. Agrega los datos llamando a aggregateData().
25. Genera la paleta de colores llamando a generateColors().
26. Crea los 6 gráficos (ver sección 5).
27. Desactiva el overlay de carga.

4.3 Detección Automática de Columnas

El sistema implementa un algoritmo de detección heurística para identificar las columnas de categoría (catCol) y de valor numérico (numCol) sin requerir configuración del usuario:

```
const headers   = Object.keys(data[0]);
const numCol    = headers.find(h => typeof data[0][h] === 'number') || headers[1];
const catCol    = headers.find(h => typeof data[0][h] !== 'number') || headers[0];
```

La lógica es la siguiente:

- numCol: Se selecciona la primera columna cuyo valor en la primera fila sea de tipo number. Si ninguna cumple, se toma headers[1] (segunda columna).
- catCol: Se selecciona la primera columna cuyo valor no sea de tipo number. Si ninguna cumple, se toma headers[0] (primera columna).

[Nota] Esta detección evalúa únicamente la primera fila del dataset. Si los datos tienen valores inconsistentes de tipo en las primeras filas, la detección puede ser incorrecta.

4.4 Función aggregateData(data, catCol, numCol)

Agrupa y suma los valores numéricos por categoría mediante un objeto de acumulación:

```
const res = {};
data.forEach(r => {
    const k = r[catCol] || 'Sin categoría';
    const v = Number(r[numCol]) || 0;
    res[k] = (res[k] || 0) + v;
});
return { labels: Object.keys(res), values: Object.values(res) };
```

El resultado es un objeto { labels: string[], values: number[] } donde cada posición del array corresponde a una categoría única con su suma acumulada.

4.5 Función getTopN(agg, n)

Ordena las categorías por valor descendente y retorna únicamente las primeras n entradas. Se utiliza para los gráficos de tipo Pie y Doughnut, que limitan a 10 y 6 categorías respectivamente para mantener legibilidad.

4.6 Función createHistogram(data, col)

Implementa un histograma de frecuencias con 8 intervalos (bins) de ancho uniforme:

28. Extrae todos los valores numéricos válidos de la columna col.
29. Calcula el mínimo, máximo y el paso por intervalo: step = (max - min) / 8.
30. Para cada bin, cuenta los valores v tal que low <= v < high.
31. Las etiquetas del eje X representan el rango de cada intervalo: "[min]–[max]".

4.7 Función generateColors(n)

Retorna un array de n colores RGBA seleccionados cíclicamente desde una paleta fija de 8 colores institucionales:

```
rgba(59, 130, 246, 1)   // Azul
rgba(16, 185, 129, 1)   // Verde esmeralda
rgba(245, 158, 11, 1)   // Ámbar
rgba(239, 68, 68, 1)    // Rojo
rgba(139, 92, 246, 1)   // Violeta
rgba(236, 72, 153, 1)   // Rosa
rgba(6, 182, 212, 1)    // Cian
rgba(249, 115, 22, 1)   // Naranja
```

5. Motor de Visualización

5.1 Función createChart(id, type, labels, values, title, color, dlConfig, fill)

Esta es la función fábrica central de gráficos. Recibe los siguientes parámetros:

Parámetro	Tipo	Descripción
id	string	ID del elemento <canvas> en el DOM
type	string	Tipo de Chart.js: 'bar', 'line', 'pie', 'doughnut'
labels	string[]	Etiquetas del eje X o segmentos del gráfico
values	number[]	Datos numéricos correspondientes a las etiquetas
title	string	Texto del dataset (aparece en tooltip)
color	string|array	Color único o array de colores (para pie/doughnut)
dlConfig	object	Configuración del plugin chartjs-datalabels
fill	boolean	Activa relleno bajo la línea (gráfico de área)

5.2 Catálogo de Gráficas

5.2.1 Gráfico de Barras (Bar Chart)

Muestra la distribución total de valores por categoría. Utiliza la suma agregada de numCol agrupada por catCol. El color de las barras usa el primer color de la paleta con opacidad 0.7.

5.2.2 Gráfico de Pastel (Pie Chart)

Representa la composición porcentual de las categorías. Se limita al Top 10 por valor para mantener legibilidad. Las etiquetas internas muestran el porcentaje calculado dinámicamente: Math.round((v / sum) * 100) + "%".

5.2.3 Gráfico de Línea (Line Chart)

Visualiza la evolución o tendencia de los valores en el orden en que aparecen las categorías tras la agregación. Incluye puntos de datos (pointRadius: 4) y curva suavizada (tension: 0.4).

5.2.4 Histograma

Muestra la distribución de frecuencias de la columna numérica en 8 intervalos de igual anchura. Las barras son adyacentes (barPercentage: 1, categoryPercentage: 1), lo cual es el comportamiento estándar de un histograma estadístico.

5.2.5 Gráfico de Anillo (Doughnut Chart)

Similar al Pie Chart pero con un espacio central vacío, lo que facilita la lectura de proporciones relativas. Se limita al Top 6 de categorías.

5.2.6 Gráfico de Área (Area Chart)

Es un Gráfico de Línea con fill: true, lo que rellena el área bajo la curva. Representa visualmente el crecimiento acumulado. Utiliza el tercer color de la paleta con opacidad reducida para el relleno.

5.3 Configuración Compartida de Etiquetas (commonDatalabels)

Todas las gráficas comparten la siguiente configuración base para las etiquetas de valor:

```
display:    Solo si el valor es > 0
color:      #1e293b (texto oscuro para contraste)
font:       weight 600, size 10px
formatter:  value.toLocaleString() — formato local de número
anchor:     'end' — posición en el extremo superior de la barra
align:      'top'
offset:     2px
```

Los gráficos circulares (Pie, Doughnut) sobrescriben anchor y align a 'center' y color a blanco o gris según legibilidad.

6. Formatos de Archivo Soportados

6.1 Especificaciones de Compatibilidad

Formato	Extensión	Versiones	Observaciones
Excel Open XML	.xlsx	Excel 2007 y superior	Formato principal recomendado
Excel Binario	.xls	Excel 97–2003	Soporte legacy mediante SheetJS
CSV	.csv	Cualquier versión	Sin soporte de múltiples hojas

6.2 Limitaciones de Archivo

- Tamaño máximo: 25 MB (limitación declarada en la UI; no hay validación programática de tamaño).
- Múltiples hojas: Soportadas en .xlsx y .xls. En .csv sólo existe una hoja.
- Tipos de celda soportados: Texto (string) y Número (number). Fórmulas son evaluadas a su valor por SheetJS.
- Fechas: SheetJS convierte automáticamente fechas de Excel a número serial. Sin conversión adicional en el código actual.

6.3 Procesamiento Interno por SheetJS

SheetJS recibe el archivo como ArrayBuffer y lo parsea con la opción { type: 'array' }. La función XLSX.utils.sheet_to_json() convierte la hoja activa en un array de objetos JavaScript, donde cada objeto representa una fila y sus propiedades corresponden a los encabezados de la primera fila.

[Advertencia] Si la primera fila no contiene encabezados de texto sino datos, SheetJS usará los índices de columna como claves, lo que afectará la detección automática de columnas.

7. Rendimiento y Limitaciones Conocidas

7.1 Comportamiento con Grandes Volúmenes de Datos

Rango de Filas	Rendimiento Esperado	Observaciones
< 1,000	Excelente (< 1s)	Sin restricciones de funcionalidad
1,000 – 10,000	Bueno (1–3s)	Pie/Doughnut muestran sólo Top N
10,000 – 50,000	Moderado (3–8s)	Posible bloqueo del hilo principal
> 50,000	Degradado	Riesgo de crash en pestaña del navegador

7.2 Limitaciones Técnicas

- Single-thread: Todo el procesamiento ocurre en el hilo principal del navegador (no se usan Web Workers). Archivos muy grandes pueden bloquear la UI.
- Sin paginación: La función aggregateData() procesa todas las filas en memoria; no existe paginación ni streaming de datos.
- Detección de columnas frágil: Depende del tipo del primer registro. Datos mixtos o headers faltantes causan detección incorrecta.
- Sin persistencia: No hay localStorage, IndexedDB ni cookies. Recargar la página borra todo el estado.
- CDN dependiente: La aplicación requiere conexión a internet para cargar las librerías externas. Sin CDN, la app no funciona.
- Sin validación de tamaño: El límite de 25 MB indicado en la UI no está programáticamente validado; el navegador puede crashear con archivos muy grandes.
- Gráficos no exportables: No existe funcionalidad de exportación de gráficas a imagen o PDF.

7.3 Compatibilidad de Navegadores

Navegador	Soporte	Notas
Chrome / Edge (Chromium)	Completo	Entorno de desarrollo y pruebas principal
Firefox	Completo	Compatible con todas las APIs utilizadas
Safari (macOS / iOS)	Parcial	backdrop-filter puede requerir prefijo
Internet Explorer	No soportado	Ausencia de Fetch API y Canvas avanzado

8. Guía de Mantenimiento y Extensibilidad

8.1 Actualización de Dependencias

Para actualizar las librerías CDN, se deben modificar las URLs en el bloque <head>. Considerar las siguientes precauciones:

- SheetJS: Verificar compatibilidad de la API XLSX.utils.sheet_to_json() entre versiones.
- Chart.js: La API de plugins (datalabels) puede cambiar entre versiones mayores. Verificar changelog.
- chartjs-plugin-datalabels: Debe ser compatible con la versión de Chart.js utilizada.

8.2 Agregar un Nuevo Tipo de Gráfico

Para añadir una séptima visualización:

32. Añadir una nueva tarjeta .chart-card con su elemento <canvas id="newChart"> en el #chartsGrid del HTML.
33. Añadir la lógica de creación en renderCharts() después de la línea del gráfico de área:

```
chartInstances.newChart = createChart('newChart', 'radar', agg.labels, agg.values,
numCol, palette[3], commonDatalabels);
```

34. El objeto chartInstances debe incluir la nueva clave para garantizar su destrucción en el próximo ciclo.

8.3 Soporte Multi-Columna Numérica

Actualmente el sistema selecciona una única columna numérica para todas las visualizaciones. Para soportar selección múltiple:

35. Modificar la detección para retornar un array de columnas numéricas.
36. Añadir un control de selección (checkboxes o multi-select) en el panel #sheetSelection.
37. Actualizar renderCharts() para generar datasets múltiples por gráfico.

8.4 Mejoras Recomendadas de Seguridad y UX

- Implementar validación programática de tamaño de archivo (> 25 MB → rechazo con mensaje claro).
- Migrar el procesamiento de SheetJS a un Web Worker para evitar bloqueo del hilo principal.
- Añadir soporte offline mediante un Service Worker para cachear las dependencias CDN.
- Implementar exportación de gráficas con canvas.toDataURL() o una librería como html2canvas.
- Agregar un modo de previsualización de los datos tabulares antes de generar las gráficas.

9. Glosario de Términos

Término	Definición
workbook	Objeto JavaScript generado por SheetJS que representa el archivo Excel completo, incluyendo todas sus hojas.
SheetNames	Array de strings con los nombres de cada hoja del workbook.
catCol	Columna de categoría detectada automáticamente; contiene los valores del eje X o las etiquetas de los segmentos.
numCol	Columna numérica detectada automáticamente; contiene los valores a sumar y representar.
chartInstances	Objeto global que almacena referencias a los 6 objetos Chart.js activos, para poder destruirlos en el siguiente ciclo de renderizado.
aggregateData()	Función que agrupa filas por catCol y suma los valores de numCol para cada categoría única.
Datalabels	Plugin de Chart.js que superpone etiquetas de texto con los valores numéricos directamente sobre los elementos visuales del gráfico.
DXA	Unidad de medida interna de documentos OOXML (1 DXA = 1/20 de punto tipográfico; 1 pulgada = 1440 DXA).
SPA	Single Page Application. Aplicación web que opera en una sola página HTML sin recargas de navegador.
Canvas API	API nativa de HTML5 que permite renderizado de gráficos 2D en el navegador, utilizada internamente por Chart.js.
CDN	Content Delivery Network. Red de distribución de archivos estáticos desde la que se cargan las librerías externas.

10. Historial de Cambios

Versión	Fecha	Autor	Descripción
1.0.0	Marzo 2026	Equipo de Desarrollo	Lanzamiento inicial. Soporte para .xlsx, .xls, .csv. 6 tipos de gráfica. Detección automática de columnas. UI responsive.

Para registrar nuevas versiones, añadir una fila a la tabla anterior con la versión semántica, la fecha de emisión, el responsable del cambio y una descripción concisa de las modificaciones realizadas.